《算法分析与设计》
课后作业

作业编号 作业 6

学　　号

姓　　名

专　　业

学　　院

二 0 二三年六月

第 6 章 贪心算法

总体要求：

（1） 采用贪心算法求解下列问题。

（2） 写出算法的求解思路。

（3） 任选一种语言 C/C++编程实现题目要求。

（4） 设计具体实例，分析算法的求解过程，并验证算法的正确性。

（5） 分析算法的时间复杂度。

（6） 问题（3）的程序以源代码方式提交，包括源程序及工程文件；第（2）（4）（5）部分以 Word 文档方式提交。将上述两个部分压缩后以.rar 文件提交。

思考题：

掌握优先队列（最小堆或最大堆）的原理和实现方法，能自己编程实现一个优先队列。

编程题：

题目 1：某体育比赛在甲乙两队中进行，两队参赛的人数相同，要求所有队员必须参赛且只能参加一次比赛。每次比赛，甲乙双方各派出一人与对方进行比赛，总胜出人数最多的队将获得比赛最终的胜利。为了取得比赛的胜利，教练在比赛前需要进行相应的排兵布阵。教练在排兵布阵时，往往会参考双方队员以往的成绩，并给出一个综合性的评分，如果某队队员相比另一队队员的评分高，则获胜的机率就越高。

输入要求：输入的第一行包含一个整数 n，表示甲乙双方出场的总人数。其后的 2 行，每行包含 n 个整数，分别表示甲乙两队各个队员的评分成绩。

输出要求：输出一个整数 k，表示甲队最大可能赢得的场次。

样例输入：

3
92 83 71
95 87 74

样例输出：

2

基本思路：

乙队每派出一名选手，就在甲队里寻找是否有选手比他评分更高，如果有，就在他们中选一个分数最低的与其比赛，然后删除该选手，并累加获胜场次；如果没有，就派剩下的全部选手中分数最低的与其比赛，然后删除该选手。

题目 2：有 n 个人参加一个马拉松接力游戏，游戏规定每个人可以根据自己的情况随时终止游戏并由下一个人继续接力。由于每个人的情况不同，即使同一个人也不可能在整个游戏过程中永远保持很好的状态。因此要求他们在比赛前根据每个人的情况需要制定一个接力规则，使整个比赛的时间越少越好。请编写程序帮助他们制定这样的接力方案。

输入要求：输入的第 1 行有三个整数 n,k 和 m，分别表示参加接力的人的个数，每个人最多可以跑的公里数以及接力赛的距离（以公里为单位）。其后的 n 行，每行有 k 个整数，分别表示每个人跑整数 1 公里,2 公里，….,K 公里所花费的时间（以秒为单位，整数）。游戏要求每个人都必须参加比赛，且每次只能跑到整数公里后才能换人。

输出要求：输出 1 个整数,表示这些人跑完整个接力赛最少要花多少时间。

样例输入：

3 5 10
2 4 7 11 15
3 5 8 12 16
2 5 9 14 19

样例输出：

28　（第 1 人跑 3km，第 2 人跑 4km，第 3 人跑 3km，总共耗时 28）

采用贪心策略求解的参考代码：

```
#include<iostream>
#include<stdio.h>
#include<malloc.h>
using namespace std;
int num;                    //表示人的总数，从文件中读取
int most_distance;          //每个人最多可以跑多少距离，从文件中读取
int total_distance;         //目标距离，从文件中读取
int** relay_distance;       //一个二维数组，用于记录各个人跑不同距离所需要的时间
int time;                   //记录目前已花费的时间
int dist;                   //记录目前已跑的里程
int* now_distance;          //用于记录各个人目前跑得里程数
void climb();
int main()
{
      //读取人数量，每个人跑的里程数，以及目标里程
    cin >> num >> most_distance >> total_distance;
    relay_distance = (int**)malloc(sizeof(int*) * num);
    now_distance = (int*)malloc(sizeof(int) * num);
    for (int i = 0; i < num; i++){
        relay_distance[i] = (int*)malloc(sizeof(int) * most_distance);
    }
    for (int i = 0; i < num; i++) {
        for (int j = 0; j < most_distance; j++){
```

```
            cin >> relay_distance[i][j];
        }
    }
    climb();
    for (int i = 0; i < num; i++){
        cout << now_distance[i] << ' ';   //将最后的结果输出到文件
    }
    cout << endl << time;
}

void climb()
{
    time = 0;
    //由于每个人都应该参与接力，因此每个人先走 1KM，计算出对应的时间和
    dist = num;
    for (int i = 0; i < num; i++)  {
        time += relay_distance[i][0];
        now_distance[i] = 1;
    }
    //在未走到最后距离时，循环搜索每个人走的距离
    while (dist < total_distance) {
        int min = 1000;     //记录各个人当前再增加 1km 需要增加的最小的时间
        int rec;            //记录达到最小时间的人的序号
        for (int i = 0; i < num; i++)  //找到在增加 1km，所需增加的时间最小的人
        {
            if (relay_distance[i][now_distance[i]] - relay_distance[i][now_distance[i]
- 1] < min && now_distance[i] < most_distance) {
                //if 语句判断该人增加的时间是否比已记录的最小时间小，并且该人跑的距离
不能超过最长距离
                min         =         relay_distance[i][now_distance[i]]         -
relay_distance[i][now_distance[i] - 1];
                rec = i;    //记录下第 i 个人有最小的时间
            }
        }
        dist++;   //增加 1km
        time          +=          relay_distance[rec][now_distance[rec]]          -
relay_distance[rec][now_distance[rec] - 1];
        //加上当前里程记录的人增加 1km 所需要的最小时间
        now_distance[rec]++;  //该人的里程增加 1km
    }
}
```

题目 3. 从一个由 k 位数字构成的正整数 n 中删除 m（m<=k）位数字，使删除后的数最大。

输入要求：

输入一行，包含两个正整数 n 和 m，之间用空格隔开。

输出要求：

输出一行，即从 n 中删除 m 位数字之后的最大数。

样例输入：

134652334 4

样例输出：

65334

参考贪心算法实验